REPUBLICA BOLIVARIANA DE VENEZUELA

MINISTERIO DEL PODER POPULAR PARA LA DEFENSA

UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA DE LA FUERZA

ARMADA NACIONAL

UNEFA

Ingeniería de Sistemas
Lenguajes de Programación II
Profesor Jesus Piñate

Ejercicios prácticos / Exposición de codigo – Unidad 1

1. Motor de Cálculo de Ranking Elo (Ajedrez)

Desarrollar el núcleo lógico para actualizar las puntuaciones de jugadores de ajedrez tras un torneo, basándose en un algoritmo matemático puramente estructurado.

Requerimientos:

- **Lógica Principal:** Implementa una función que reciba el Elo de dos jugadores y el resultado de la partida (1, 0, o 1/2) y devuelva los nuevos puntajes. Esto requerirá uso intensivo de operaciones matemáticas básicas y estructuras de control condicionales. Tengo que poder acceder al historial de partidas que actualmente se encuentran en memoria. Implementando un buscador de partida y de registros de jugador y cualquier otro criterio que usted vea bueno.

- **Procesamiento por Lotes:** Crea un flujo que lea una serie de partidas con el formato JugadorA | JugadorB | Resultado, y procese múltiples partidas en bucle, actualizando un diccionario en memoria con los puntajes acumulados.

- **Manejo de Errores:**

    - Crea una excepción *JugadorNoEncontradoError*.
    - Implementa un bloque try/except que atrape líneas mal formateadas en el input. Si una línea está corrupta, el programa debe registrar el error pero continuar iterando con el resto de las partidas.

2. Simulador de Máquina Expendedora con Máquina de Estados

Emula el funcionamiento interno de una máquina expendedora gestionando el inventario y el flujo de caja mediante una arquitectura altamente modular.

Requerimientos:

- **Estructuras de Datos:** Utiliza diccionarios anidados para manejar el inventario (producto, cantidad, precio) y las monedas disponibles para dar vuelto.
- **Modularización:** Separa la lógica en *inventario.py*, *transacciones.py* y *consola.py*.
- **Excepciones Críticas:** Debes definir e invocar (*raise*) al menos tres excepciones personalizadas: *ProductoAgotadoError, SaldoInsuficienteError y DenominacionInvalidaError*.
- **Gestión de Flujo:** Al procesar un pago, utiliza un bloque *try/except/finally*. Si el proceso de dispensar el producto falla (por ejemplo, por falta de inventario lanzada en el try), el bloque *finally* debe garantizar invariablemente que se devuelva el dinero introducido por el usuario.

3. Analizador Criptográfico de Cifrado Clásico

Crea una suite de herramientas de consola para cifrar y descifrar mensajes utilizando métodos clásicos (César y Vigenère), demostrando el dominio sobre el tipo de dato *string* y sus métodos.

Requerimientos:

- **Lógica:** Manipulación de cadenas a bajo nivel usando iteraciones (*for/while*), conversión de caracteres a sus valores ASCII (*ord(), chr()*) y aritmética modular. No se permite importar módulos de criptografía.
- **Paquetes:** Construye un paquete criptografia que contenga un submódulo para cada algoritmo y un módulo principal de enrutamiento.
- **Manejo de Errores:** Valida estrictamente las entradas. Si el usuario ingresa una clave alfabética cuando se espera una numérica (César), el programa debe atrapar el ValueError nativo, transformarlo en un mensaje amigable y volver a pedir el dato sin romper la ejecución (bucle de reintento).

4. Calculadora de Gastos de Viaje Grupal.

Crea una herramienta que ayude a un grupo de amigos a dividir los gastos de un viaje.

- **Lógica:** El programa debe permitir ingresar nombres de personas y los montos que cada uno pagó. Al final, debe calcular el gasto total y cuánto debe poner (o recibir) cada persona para que todos hayan pagado lo mismo.
- **Paquetes:**
  - *calculos.py*: Contiene la lógica para sumar el total y calcular el promedio por persona.
  - interfaz.py: Funciones para imprimir los resultados de forma bonita.
- **Manejo de Errores:**
  - Si se intenta calcular el promedio pero no se ingresó a ninguna persona, lanza una excepción *ValueError* con el mensaje "No hay participantes".
  - Usa un bloque *try/except* para manejar la división por cero (en caso de que la lista de personas esté vacía).

5. Simulador de Motor de Reservas y Overbooking

Programa el núcleo de un sistema de reservas para una aerolínea que opera en memoria. El reto no es solo guardar datos, sino gestionar la disponibilidad competitiva y la recuperación ante fallos de lógica.

Requerimientos:

1. **Estructura de Datos Compleja:**
   - Debe existir un diccionario maestro de vuelos donde cada llave sea un código de vuelo y el valor sea otro diccionario con: *capacidad_total*, *asientos_ocupados* (una lista de diccionarios de pasajeros) y *lista_de_espera*.
   - No se pueden usar bases de datos. Todo debe procesarse mediante iteración y búsqueda en estas estructuras anidadas.
2. **Arquitectura Modular:**
   - *vuelos/gestion.py*: Funciones para crear vuelos y verificar disponibilidad.
   - *vuelos/reservas.py*: Lógica para asignar asientos, manejar cancelaciones y mover pasajeros de la lista de espera al vuelo.
   - *vuelos/excepciones.py*: Definición de todos los errores personalizados.
3. **Manejo de Errores y Transacciones Simuladas:**
   - *OverbookingError*: Se lanza si se intenta reservar en un vuelo lleno (el sistema debe atraparlo y sugerir mover al pasajero a la lista_de_espera).
   - *ReservaInvalidaError*: Si los datos del pasajero están incompletos o el formato del ID es incorrecto.
   - *Uso de finally*: En el proceso de reserva, el bloque finally debe generar un "Ticket de Intento de Transacción" (un print con un código único) que se

muestre siempre, indicando si la reserva fue EXITOSA, FALLIDA o EN ESPERA.

Para cada uno de los siguientes proyectos, deberá entregar el enlace a un repositorio público en GitHub. En dicho repositorio se evaluará:

1. **Entorno Virtual:** Presencia de un archivo requirements.txt (aunque solo contenga herramientas de testing nativas o esté vacío, demostrando que el entorno fue aislado) y un .gitignore configurado correctamente para excluir la carpeta *venv/* y archivos caché de Python *(__pycache__).*
2. **Arquitectura Modular:** Ningún ejercicio puede estar escrito en un solo archivo main.py. Deben existir paquetes, módulos y una clara separación de responsabilidades (lógica de negocio vs. interfaz de usuario).
3. **Entrega:** Debera entregarse antes de la siguiente clase enviando todos los links de los repositorios del grupo al mismo tiempo. Posteriormente tendrá que dar explicaciones concernientes al código realizado.